Calibrated concept profiles: making the reading of a model's neurons a measurement

Piotr Wilam *piotrwilam@gmail.com*
University College London

Abstract

Interpretability has largely moved from reading a model's neurons directly to reading a second model trained on the first: a probe, a sparse autoencoder, a crosscoder. The move answered real faults — mixed meanings, thresholds without a unit, readings that did not transfer from one study to the next — but gave up what a direct reading alone provides: nothing trained in between, the neurons themselves named, and a reading that can be repeated and compared. Here the direct reading is made a measurement.

A neuron's activation varies with the prompt, so its reading is a distribution over a population of stimuli. We declare that population: every factor around a target construct is drawn at its rate in a reference corpus and logged.

From the declared distribution we generate a battery of prompts, and draw it twice. From the battery we compute the *calibrated concept profile*, one number per neuron, stated in units of the neuron's own spread on ordinary input; the two draws give every number a noise unit. Before the profile is read, a fixed set of checks licenses the battery on the model, and the smallest change the profile can resolve across training checkpoints is measured.

Realised for Python on OpenCoder-1.5B and followed across checkpoints spanning two trillion training tokens, the profile gives what the alternatives do not report: a unit on every number, a chance level for every comparison, and which neurons carry a construct at every checkpoint.

The battery, the code and the data behind every result are released.

1 Introduction

1.1 Why read neurons directly

Reading a model's neurons directly is the oldest tool of interpretability, and the field has largely moved past it. The reasons are known. A single neuron can mix several meanings (Elhage et al., 2022); a list of neurons has no stated threshold; and the readings did not travel from one study to the next. Learned decompositions — linear probes, sparse autoencoders (Bricken et al., 2023; Cunningham et al., 2023), crosscoders (Lindsey et al., 2024) — answered these by training a second model on the first and reading that instead. What is given up is not small: a direct reading has nothing trained in between, it names the model's own neurons, which can then be removed to test what they do, and it can be repeated — on other stimuli, at another checkpoint, on another model of the same architecture — and the results compared. This paper takes no side on whether neurons or learned features are the better unit. It claims that a direct reading can be made a measurement — of a declared population of stimuli, in a stated unit, under a licence that says when it may be used, at a stated resolution — and shows what such a measurement sees (§4, §5.1).

1.2 What a reading is

A neuron, as the word is used here, is one unit of a transformer's MLP block: at one position of the input it receives the residual stream, applies its weight vector and a nonlinearity, and outputs one number, its

Table 1: The factors of a stimulus, and what each approach does with them.

factor of a stimulus	template	corpus sample	declared distribution
the target construct in its role	fixed	present, uncontrolled	fixed (set A)
the target's token out of role	rarely provided	uncontrolled	fixed (set B)
identifiers and literals	fixed by the template	uncontrolled	drawn at corpus rates, logged
scope and nesting	fixed	uncontrolled	drawn at corpus rates, logged
companion constructs	fixed or absent	uncontrolled	drawn at corpus rates, matched between A and B, logged
length and surrounding code	fixed	uncontrolled	drawn at corpus rates, logged
ordinary behaviour of a neuron	not measured	not separated	measured on ordinary code (set C), per kind of position
what a repeated draw gives	the same prompts	another sample	a second battery: the noise unit

activation. Reading a concept from neurons means two things: a set of inputs shown to the model, the *stimuli*, and a rule that turns the activations they produce into one statement per neuron, the *arithmetic*. For a concept such as the `for` loop, the stimuli are prompts that contain `for` loops, and the arithmetic says, for each of the model's neurons, how much it responds to those loops. The question is the one asked since the first inspections of hidden units (Karpathy et al., 2016; Bau et al., 2017): which neurons respond to a given concept across many inputs, with whatever one prompt adds averaged out. The readings of named concepts in the literature — knowledge neurons (Dai et al., 2022), language-specific neurons (Tang et al., 2024), universal neurons (Gurnee et al., 2024), the neuron readings of code models (Sharma et al., 2023; Kargaran et al., 2025; Yin et al., 2025) — all have this form.

1.3 A reading is a distribution, over a population someone chose

Two facts about the form decide what such a reading is. A neuron's activation is a continuous value, not a yes or no. And the stimuli vary: two prompts containing a `for` loop differ in their identifiers, their nesting, their length and in what stands near the loop, and the activation differs with them. So the reading of a neuron is a distribution: one value for each stimulus in some population, with a mean and a spread.

That population is the researcher's choice, and two ways of choosing it dominate. *A template*, a handful of prompts written by hand: every factor is fixed, so the reading is conditional on choices nobody declared; circuits found on templated prompts behave differently off them, and sparsity does not remove the dependence (Wu et al., 2026; Tan et al., 2024). *A sample from a corpus*, passages of real text that contain the construct: nothing is fixed, so the population cannot be described, it may overlap the model's training data, and there is nothing to subtract. The first is too rigid, the second too loose.

Table 1 sets the two ways against the declared distribution of §2.1, factor by factor. The template column is the design of the minimal-pair benchmarks (Warstadt et al., 2020; Gauthier et al., 2020). The corpus-sample column is the design of studies that read neurons over passages of real text, such as the language-specific neurons of Tang et al. (2024) and the universal neurons of Gurnee et al. (2024).

1.4 What a measurement must state

We take the position that, once the population is a choice, a reading counts as a measurement only if it states four things: the population, declared; the unit, the spread of the reading when the stimuli are drawn again from that population; evidence, from checks run before use, that its numbers come from the model's response and not from the stimuli or the arithmetic alone; and, for comparisons across training checkpoints, the smallest change it can resolve. Current neuron-level readings do not state them: thresholds without a unit, one checkpoint, and standardisation against pooled baselines although neurons are positional and token-class specific (Voita et al., 2024). The variance of circuit discovery has been measured (Méloux et al., 2025; Wu et al., 2026) and circuit components certified against resampling (Anani et al., 2026), but not for neuron readings of named concepts.

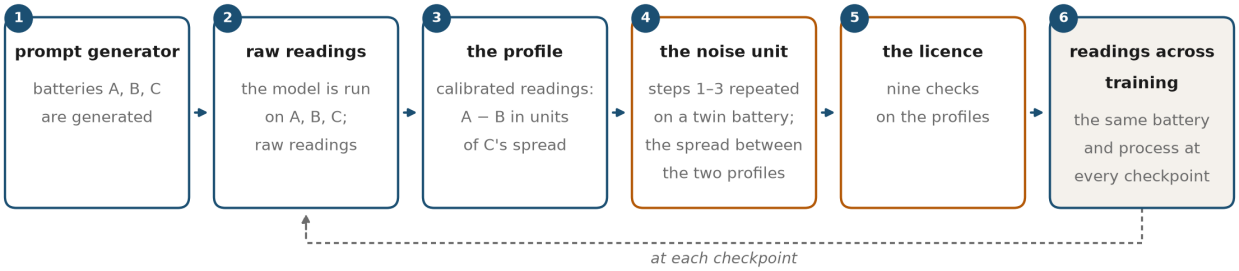


Figure 1: The calibrated concept profile in six steps (§2.1–2.6), for one construct, at each checkpoint.

1.5 The solution

The solution has two parts, one for each step of a reading.

1. *Controlled stimuli.* The prompts are generated, not collected or written by hand. A declared distribution lists every property around the target — identifiers, nesting, length, the constructs nearby (Table 1) — and draws each at the rate a reference corpus shows; the target is held fixed, everything else is as ordinary input of the domain has it, and every draw is logged, so that the population can be stated.
2. *A reading that keeps the distribution.* A neuron's activations over the prompts are treated as a distribution, not as a value. The reading takes their mean, states it in units of the neuron's own spread on ordinary input, and reports how far it moves when the prompts are drawn again.

We call the reading the *calibrated concept profile* of a construct, the profile for short, and the method of making and licensing it goes by the same name.

2 The calibrated concept profile

The method is built in six steps (Figure 1). Each step answers a question that the previous step leaves open; §3 realises the six for Python on OpenCoder-1.5B.

2.1 Step 1 – prompt generator

Which prompts should be shown to the model to read a construct, and how should they be chosen so that the reading can be repeated? A *declared stimulus distribution* for a target is a written list of the factors around the target — for code: identifiers and literals, scope and nesting, companion constructs, length and surrounding text — and, for each, the rates at which its values are drawn, measured in a reference corpus. A prompt is one draw from the list; the draw is logged with the prompt. Nothing about a prompt is chosen for a reading. Three sets are drawn from the list, differing in what is held fixed:

1. *Set A* holds the target in its role and draws every other factor at its corpus rate.
2. *Set B* holds the target's token outside its role — inside a string, a comment, a docstring, or as part of a name — and, where the token also has another grammatical role (the `for` of a comprehension, say), in that other role. Its companions are drawn so that their mix matches A's, in distribution rather than prompt by prompt, so that A and B differ only in the target's role; counterfactual datasets reach the same end one text at a time, by rewriting (Abraham et al., 2022). B does for the reading what a control task does for a probe (Hewitt & Liang, 2019): it removes what the token alone accounts for.
3. *Set C* is the baseline: ordinary input of the domain, drawn at the corpus rates with no regard to any target. It is what a neuron's ordinary behaviour is measured on (§2.3).

Two properties of the declaration are used later: the logged draws can be re-weighted to another distribution by arithmetic, without running the model again; and the list can be drawn from twice (§2.4).

2.2 Step 2 – raw readings

A neuron's activation at the construct's position mixes several sources. How is the part due to the construct's role separated from the rest? The reading is made at one position per prompt, the *read site*: the target's first token. A neuron's activation there has four sources. One is what we want, the target's role. Three are what we want to remove: the particular prompt; the token, which contributes the same whatever other role it is in; and the neuron's ordinary level and scale at that kind of position. Averaging over A removes the prompt: the mean activation per neuron over set A keeps what the prompts share, the role. Subtracting B removes the token: set B carries the same token and the same companions, read at its first token, so the mean over B holds what the token does on its own. The difference, mean over A minus mean over B, is the neuron's response to the role. Taken on the activations as the model produces them, we call it the *raw form* of the profile. It is still in each neuron's own units: neurons differ in their ordinary level, one may sit near 0 and another near 3, and in their ordinary spread, one varying by tenths and another by tens, so a raw difference of 0.5 is large for one neuron and nothing for another. The next step restates it in each neuron's own spread.

2.3 Step 3 – the profile

In what unit should a neuron's response be stated, so that neurons with different ordinary levels and spreads can be compared? In the neuron's own ordinary spread. For every neuron and every kind of position, μ_0 and σ_0 are the mean and the spread of its activation over set C; an activation x becomes $z = (x - \mu_0)/\sigma_0$. The pair (μ_0, σ_0) per neuron and kind of position is the *yardstick*, and the *profile* in its standardised form is the difference of §2.2 taken on z. An entry of 2 says: this neuron responds to the role by two of its ordinary spreads more than to the token alone.

The unit is taken per kind of position because a neuron's ordinary behaviour is not one number: its level and spread depend on where it is read. Take `len`, read at its name, in `n = len(x)` on the first line of a short prompt and in `for k in range(0, len(x)):` twenty lines in. The token is the same; the position is not, and a neuron's ordinary level and spread differ between the two, as they differ between a keyword and an identifier (Voita et al., 2024). A single pair taken over all positions would make the unit wrong at both. A *kind of position* is therefore a class of places in a prompt within which one pair is measured; which classes are distinguished is a choice of the realisation (§3.3). One condition follows: A's and B's read sites must fall in the same kinds, so that μ_0 cancels in the difference and σ_0 alone does the work.

Both forms of the profile are kept. The standardised form says which neurons carry the construct and how concentrated the response is; the raw form gives its size, because σ_0 itself changes over training and a standardised size would confound the two. Set C enters only through σ_0: it ties the unit to the reference corpus, which stands in for the model's training distribution (§6).

2.4 Step 4 – the noise unit

How much does the reading change when the prompts are drawn again from the same distribution, that is, how large is its noise? Every prompt is built from one of a few structurally distinct *cores* of the target, dressed in drawn surroundings. The battery is drawn twice from the same list, on the same cores with different surroundings — the *twins*. Each twin gives a profile; their disagreement is what the surroundings alone can do. It is a noise floor in the sense of Chen et al. (2026), measured on the instrument before any effect is read against it. Because the twins share their cores this is surface noise only; variation between cores is carried by the bootstrap.

1. *The twin unit.* The distance between the two twins' profiles, $d = \sqrt{(2(1 - \cos))}$, where cos is the cosine between them. A difference between two profiles — two constructs, two checkpoints — is the same distance between them, stated in multiples of d.
2. *The core bootstrap.* An interval on any number derived from a profile comes from resampling the cores with replacement and recomputing, with the same resampled cores for both twins and for every checkpoint compared.

Table 2: The nine checks of the licence.

check	what it asks	what its outcome restricts
1. twin agreement	do two independently drawn batteries give the same profile?	any statement about which neurons carry the construct
2. convergence	after how many prompts does the profile stop changing?	readings from fewer prompts than the convergence point
3. length	does lengthening a prompt change the profile?	comparisons across prompts of different length, for the sensitive constructs
4. re-weighting	does re-weighting the logged factors to other proportions change the profile?	comparisons across different distributions
5. calibration	do the stated intervals cover the twin difference at their nominal rate?	the width of intervals: inflated by a stated factor
6. companion	do A and B carry the same companion constructs, within a cap?	statements about the construct alone: its profile includes the companion difference
7a. scrambled null	with A and B labels shuffled, does a profile remain?	a gate: the real profile must exceed the null's 95th percentile, or nothing is read
7b. split-half null	do two halves of the same set give a "profile"?	a gate, as 7a
8. yardstick floor	how far does the floor on σ_0 move the reading?	readings of neurons whose σ_0 sits at the floor

No distributional form is assumed: the yardstick is a change of unit, intervals come from the core bootstrap, and thresholds are stated in noise units, not as significance levels.

2.5 Step 5 – the licence

Before a model is read, how do we know the instrument works on it? It may be used on a model when a fixed set of nine checks has run on the pair (battery, model): the two with a pass criterion have passed, and the other seven have been reported. A new battery, or a new model, needs a new licence. The checks measure the instrument, not the model's learning.

The two nulls follow the corruption check of Oikarinen et al. (2025) and the random-set null of Kim et al. (2018), applied to an instrument rather than to a metric. Checks 2, 3, 7a and 7b need the activation of each single prompt, since they add, pad or relabel prompts one at a time; the others work on the stored means.

The nine checks are of two kinds. The two nulls are a gate: if either fails, the profile cannot be told from what the arithmetic or the sampling produce alone, and nothing should be read. The other seven are not a gate. Each measures one way the reading depends on the particular battery, corpus or model — on the draw (check 1), the number of prompts (2), prompt length (3), the mixture of logged factors (4), the stated intervals (5), the companions (6), the yardstick's floor (8). Their outcomes do not decide whether the instrument may be used; they state the restrictions within which it is used, and which experiments are safe to pursue within them is the researcher's decision. A profile that moves with the mixture is compared only over the same distribution; a length-sensitive construct is not compared across prompts of different lengths; a construct whose A and B differ in companions carries a companion part in its profile. The last column of Table 2 names the claims each outcome restricts. §4.1 gives the restrictions measured on OpenCoder, and the claims of §4 are made within them.

2.6 Step 6 – readings across training

Which numbers are read from the profile, what are they compared with, to tell the model's response from what the battery and the architecture produce on their own, and how small a change between checkpoints can be detected? Three numbers, then one set and its ceiling:

1. *Norm* — the size of the response, on the raw form.
2. *Sparsity* — the share of the squared norm in the largest 5% of entries, on the standardised form: how concentrated the response is.
3. *Twin agreement* — the cosine between the two twins' profiles, from which the twin unit is computed; read as a property of the construct, how repeatable the response is.

Table 3: The six choices, what depends on them, and how far each is measured.

choice	what depends on it	how far it is measured
1. *Two twins*: two draws of the battery, sharing their cores	every statement "beyond noise"; the twin ceiling; the resolution	twin agreement (check 1); intervals from the core bootstrap (§4.8). Not measured: more draws, or draws on independent cores
2. *The generator*: cores and axes, menus of names, how B is built, companion matching	what the profile is a profile of	the mixture (check 4: median 2.34 twin units); companions (check 6: not matched for compound statements). Not measured: generated prompts against real code
3. *The yardstick*: token class × fifth of the prompt; set C; the floor on σ_0	which neurons are named; size in standardised units	coarser yardsticks name different neurons (§4.3); the floor (check 8); length (check 3). Not measured: other splits, computed exactly
4. *The reference corpus*: public Python, not the training data	what "ordinary" and "at corpus rate" mean	not measured: how far public Python differs from the model's 607-language mixture
5. *Naming rules*: the inventory, the read site, the gate, the top-5% carrier set	which constructs are read; which neurons are the carriers	the gate's failures (19 of 188, §3.1); the cut does not change carrier turnover (Wilam, 2026). Not measured: another read site
6. *The read point*: MLP neurons, one position, short prompts (median 55 tokens)	everything read	not measured: longer contexts, attention, the residual stream

4. *The carrier set* — the 5% of neurons with the largest standardised entries. Two random 5% sets overlap by 0.05.
5. *The twin ceiling* — the overlap of the two twins' carrier sets, the most two batteries can agree.

The baseline is the same architecture with random weights, read with the same battery: whatever it gives is what the battery and the architecture give on their own. A construct is *concentrated* if its sparsity exceeds the untrained value by more than three noise units and its twin agreement by at least 0.20; both margins were fixed before any reading. Two profiles of one architecture — two checkpoints, two runs, a trained model and its untrained reference — are compared in twin units: the distance between them, or the overlap of their carrier sets against the twin ceiling. This is what lets the method follow a construct through training.

A comparison across training checkpoints needs one more number: the *resolution*, the step between a construct's profiles at neighbouring checkpoints, in twin units. Where it is below one, no date can fall between them.

2.7 The choices the method rests on

Which choices does every reading depend on, and how far is each dependence measured? Every reading is conditional on six choices. For each, Table 3 says what depends on it and how far this paper measures that dependence; §2.5's checks are the measurements, and what is not measured marks the method's open edge.

3 One realisation: Python on OpenCoder-1.5B

Three things were built, each from the one before: a battery of prompts, a record of the model's activations on it, and the calibrated concept profiles computed from the record. Each choice is one way of meeting §2; the generator's mechanics and the details are in Appendix A.

3.1 The battery

The inventory: the whole language. The constructs are not a chosen sample: every node of Python 3.12's grammar that appears as text of its own, and every public builtin name — 63 + 143 = 206 constructs, of which 188 were built. What is excluded is excluded by a rule applied to the whole grammar, not by a judgement about any particular construct (Appendix A.1).

The battery. OpenCoder's training data was not released, so a public corpus of 253,398 parsed Python files is the reference corpus, and its rates are the declared rates. Each construct has eight *cores* drawn from a family of between 42 and 1,280 (median 210), and each of A and B holds 720 prompts in each twin, ninety from every core, the same for all 188 constructs — 1,440 prompts a set across the two twins; companions are drawn at corpus rates and matched between A and B; every prompt is formatted, parsed and logged. For `for`, A's prompts are loops — `for x in range(n):` with a body, among drawn identifiers, at a drawn scope, with the companion statements real code puts near a loop — and B's carry the token `for` inside a string, a comment or a name — with the same companions; for the eighteen constructs whose token has another role, B also holds it in that role (Appendix A.2). The generator never runs the model and never reads the training data.

The read site and its gate. The read site is the first token of the construct's node: on `for x in range(n):` the token `for`; for a call to `len` the name `len`. Every recorded quantity carries the prompt's scope, because a statement-initial keyword is a different token indented (␣`for`) and at module level (`for`), two embeddings as unrelated as a random pair (cosine 0.056). A *read-site gate* admits a construct only if the most common token piece at its read site covers at least 80% of prompts, counted on the indented scopes, in both twins and on both A and B: 169 of 188 pass; these form P_1, the population read in §4. The nineteen failures are listed in Appendix A.4. The gate is the one place where the tokenizer, not the grammar, decides what can be read.

Artefact: the battery in full — the prompts of A and B for all 188 constructs in both twins, the set C, set A regenerated for every construct from a second declared distribution, sixty prompts each — check 4's alternative menu, not a further draw from the same menu, which is what a twin is — and the log of every draw — with its hash (`570b42db...`), in the dataset `piotrwilam/AtlasOpenCoder`, tag `v1`.

3.2 The record

The model and its neurons. OpenCoder-1.5B (Huang et al., 2024) has 24 layers with an MLP hidden width of 6,144 (SwiGLU). Each MLP block's hidden layer has 6,144 units; these are the neurons. A unit's activation is the product of its gate branch, after the nonlinearity, and its up branch, taken before the down-projection returns it to the residual stream (Figure A.2); it is the quantity Geva et al. (2021) read as a key-value memory. All $24 \times 6{,}144 = 147{,}456$ neurons together are the space in which comparisons across checkpoints are made, which we call the *product space*.

The references. Sixty-one references were shown the battery: 35 pretraining checkpoints (the twenty from 4.19 to 83.9 billion tokens, then fifteen spaced $\times 1.25$ to 2.03 trillion), 24 annealing checkpoints and two untrained references. The per-core means are stored, with the cell axis kept, so that the size of the carrier set can be changed later, and the yardstick by merging cells, without re-recording (Appendix A.6).

What is stored per prompt. The per-core means are enough for every reading in §4 and for checks 1, 4, 5 and 8. Single-prompt activations, which checks 2, 3, 7a and 7b and the probe of §4.7 need, are kept at four references — the untrained model, 1,000 and 20,000 steps, and the final checkpoint: the MLP neurons for 31 constructs drawn to represent the read population, and the residual stream for 164 (Appendix A.6). Check 6 applies to the eight constructs that can have a companion: `ClassDef`, `For`, `FunctionDef`, `If`, `Match`, `Try`, `While`, `With`.

Artefact: the record is not released in full; at 27.6 GB of accumulators a reference it comes to 2,022 GB. The dataset holds a worked example — twin 1's accumulators for `for`'s A and B sets and for set C at the final checkpoint (272 MB) — and the recorder in the repository records any reference again from the model's public checkpoints and the battery.

3.3 The profiles

The cells. A kind of position (§2.3) is here a *cell*, defined by two properties of a position: the class of the token at it — keyword, identifier, symbol, newline or indent — and the band it stands in, which fifth of the prompt's length. Four classes times five bands gives 20 cells, and $20 \times 147{,}456 = 2.95$ million (μ_0, σ_0) pairs per reference. The token class is the property a neuron's ordinary behaviour depends on (Voita et al., 2024;

§2.3). The band is relative to the prompt's length rather than absolute for two reasons: every cell of C is then populated across prompts of 2 to 313 tokens, and A's and B's read sites fall in the same cells, as §2.3 requires. The cost is that the same prompt, lengthened, can move its read site to a different cell (§4.1).

From the record to a profile. (1) On set C's part of the record, for each neuron and each of the 20 cells, the mean μ_0 and spread σ_0 of the activations at tokens in that cell: the yardstick, which knows nothing about any construct. (2) For each A prompt of a construct, the read site's cell is found from its token class and band — for a `for` loop at an indented scope, a keyword in the band of the loop's line — and each neuron's activation there is standardised with that cell's pair. (3) The same for each B prompt. (4) For each neuron, the mean over A minus the mean over B: the profile, one number per neuron; the same on the raw activations gives the raw form. The standardisation is applied to each prompt's activation before averaging, with the pair of that prompt's own cell; the two forms are therefore two different averages, not one average rescaled. (5) Steps 2–4 run on both twins; from the two profiles come the twin unit, the carrier set (7,373 neurons) and its twin ceiling, and the core bootstrap's intervals (§2.4, §2.6).

Artefact: the carrier sets of every construct at all 61 references, with the twin ceilings and the licence tables of Appendix B; and, as a worked example, the profile of `for` in twin 1 at the final checkpoint, every array. The other profiles follow from the record.

4 The instance: what the battery on OpenCoder shows

§1.4 said what a reading must state to count as a measurement. The population is §3.1's. §4.1–4.4 state the other three for this instance: the evidence that the numbers are the model's (the licence, §4.1; the ablation, §4.4), the resolution (§4.2) and the unit's load (§4.3); §4.5–4.8 show what the instrument then sees.

4.1 The licence as run

Does the instrument work on OpenCoder-1.5B — do its nulls pass, and how stable, convergent and calibrated is its reading? On the terms of Table 4: both nulls pass with a wide margin for every construct they ran on, so the gate is passed; the seven other checks show a stable, convergent reading, calibrated at the median, with three restrictions, from checks 3, 4 and 6, stated below and in the licence sentence.

Table 4: The licence on OpenCoder-1.5B, final checkpoint.

#	check	ran on	outcome
1	twin agreement	188	median cosine 0.986 between the twins' profiles
2	convergence	31	cosine 0.99 with the full set reached by 56 of 720 prompts (median)
3	length	31	a median 2.4% of a construct's neurons length-sensitive; more than 5% for eight statements (`With` 55%, `For` 49%)
4	re-weighting	188	designed arm only; the re-weighted profile sits a median 2.34 twin units from the base, within one twin unit for 4 of 188 — the profile moves with the mixture
5	calibration	188	calibrated at the median; intervals optimistic for 33 constructs, by a stated factor
6	companion	8	the generator's matching is not met: the largest difference between A's and B's share of a companion is 0.19–0.68 (median 0.37) against the design's cap of 0.15; removing the difference, where it can be removed, moves the profile by 2.95 twin units (`For`) and 5.50 (`With`)
7a	scrambled null	31	real profile a median 10.7× the null's 95th percentile (minimum 7.2×); 0 of 31 at or below — pass
7b	split-half null	31	median 12.8× (minimum 6.0×); 0 of 31 at or below — pass
8	yardstick floor	188	moves a profile by a median 0.0039 and at most 0.035 twin units

Check 4 is the reason the distribution is declared (§2.5): re-weighting the logged factors moves the profile by more than twice what redrawing the battery does. Length sensitivity is partly the yardstick's doing (Appendix B.4); it does not affect comparisons across checkpoints, because every checkpoint sees the same prompts. Check 6 restricts what can be said about the eight compound statements: their B sets do not carry the same companion constructs as A (Table B.3), so their profiles include a companion part. Where

B can be re-weighted to A's companion shares — `For` and `With` — the profile moves by 2.95 and 5.50 twin units, of the order of check 4's re-weighting. The companion part does not change the turnover: between every pair of the four per-prompt references, the carrier overlap of `For` and `With` moves by at most 0.008 when B is re-weighted, so comparisons across checkpoints are not affected. It changes which neurons are named for `With` but not for `For`, and the re-weighting is not adopted as the reported profile (Appendix B.3). Per-construct values are in Appendix B.

The licence sentence: licensed on the gate — the two nulls passed on the 31 constructs they can run on — with these restrictions: the profile moves with the mixture by a median 2.34 twin units (check 4, designed arm only); eight statements are length-sensitive (check 3); for the eight compound statements A and B differ in companions, by 2.95 and 5.50 twin units where measurable (check 6); 33 constructs' intervals are optimistic by a stated factor (check 5); the second-model replication outstanding and named.

4.2 The resolution, and a construct followed through training

How small a change in a construct's profile between two checkpoints can the record detect, and so how finely can it date a change? To the spacing of the checkpoints, 1,000 training steps: even at that spacing the step exceeds one twin unit, so the instrument's own limit lies below it. On P_1 the median step between neighbouring references is 8.56 twin units between the first two checkpoints (1,000 steps apart) and 3.56 at the end of the dense block; 4.06 across the late tail and 2.91 on the annealing branch. The minimum is above one everywhere except `None`, at 0.862 and 0.918 in two pairs. At the same 1,000-step spacing early training moves the profile further than late: 8.56 twin units against 3.56, a ratio of 2.4.

The same profile, read at 61 references, follows one construct from the first checkpoint to the end. For `for`, the raw norm ends at 52% of its first-checkpoint value while the standardised norm does not fall, which is why size is read on the raw form (§2.3, §4.8; Appendix C.2). Wilam (2026) reads the whole record this way, for every construct, at this resolution.

4.3 The unit is load-bearing

Does the choice of yardstick change which neurons are named? Yes: the yardstick is part of the measurement. Change what counts as ordinary at a position and the named neurons change by more than redrawing the battery changes them. Under coarser yardsticks the carrier sets overlap the 20-cell sets by a median 0.73–0.87, below the twin ceiling for 78 to 167 of the 169 constructs, and the token class carries more of the set than the position band. A comparison with an alternative split, registered in the design, is deferred: the stored record, kept in the 20 cells, supports a split that cuts across them exactly for only the 31 constructs with per-prompt activations, so it needs a new recording keyed by the alternative's cells.

The unit also separates the model from the battery. Twin agreement, concept norm and carrier overlap with the previous reference are 0.981–0.989, 48.7–68.8 and 0.822–0.851 on every trained reference, and 0.710 / 6.34 / 0.273 and 0.717 / 6.13 / 0.286 on the two untrained references, recorded days apart under different seeds; the twin ceiling rises from 0.28 untrained to 0.84 at the first checkpoint (plug-in; core-bootstrap means $0.21 \rightarrow 0.74$).

4.4 The named neurons carry function

Do the neurons the profile names as carriers affect the model's predictions at the construct's positions? Removing them raises the loss where the construct's grammar fixes the next token. For the 16 constructs whose prompts carry such positions, zeroing the carrier set raises the loss there by more than zeroing a random set of the same size does: by a median 0.06 nats at 4.19 B tokens [0.02, 0.14], 0.25 at 83.9 B [0.03, 0.66] and 0.40 at 2,034 B [0.10, 0.70] (construct-bootstrap 95% intervals over 200 random sets; above the random mean for 12, 14 and 14 of 16 constructs, sign test $p = 0.038$, 0.002, 0.002). The largest: `Try`, whose carriers cost 5.95 nats more than a random set of the same size, on a 0.088-nat baseline; `Lambda`, 4.99 nats more, on 1.40. The comparison is with the random sets' mean and by a sign test over constructs, not in units of the random sets' spread, which is long-tailed (Appendix C.1; Table C.1 gives the same comparison

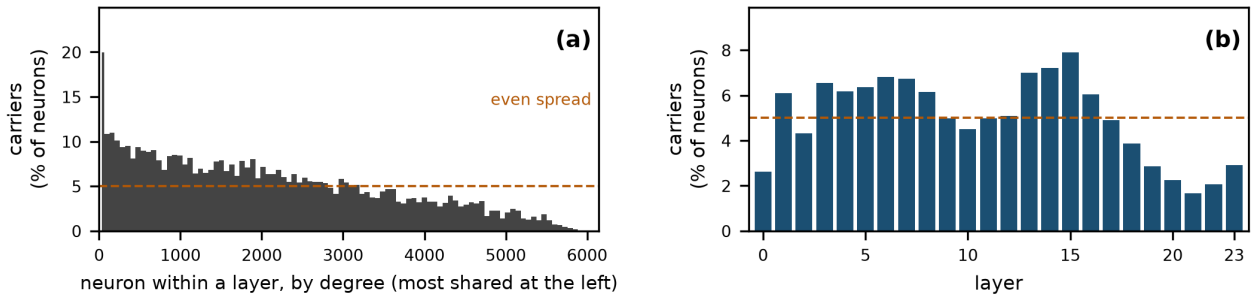


Figure 2: Where the `for` loop's carriers sit, at the end of pretraining (2,034 B tokens). (a) Carriers per unit of neurons, the neurons ordered by degree, the number of the 169 carrier sets each belongs to (high degree at the left); the dashed line is the 5% rate carriers would have if spread evenly. (b) Carriers per layer, against the 5% rate. The carrier set is concentrated on high-degree neurons and present in every layer.

under one pooled cell). At 4.19 B the median effect is a fifth of its value at 83.9 B: the neurons acquire most of their function during the first 20× of tokens.

4.5 The model's `for` is neither a token nor a role

Do the neurons that carry `for` follow the token `for`, whatever its role, or the role of a loop, whatever its token? Neither: two roles of one token share a quarter of their neurons, and one role across two tokens shares half. *One token, two roles:* `for` as loop and `for` as comprehension, separate A sets with their own cores, have carrier sets overlapping by 0.250–0.266 at the last checkpoint (0.198 for `AsyncFor`); chance is 0.050 and the ceiling 0.872, so five times chance and under a third of the ceiling. `if` as statement and as conditional expression likewise, 0.266. *One role, two tokens:* ␣`for` and `for`, the loop indented and at module level, overlap by 0.494 against the two scope-restricted sets' own twin ceilings of 0.837 (module) and 0.850 (indented), lower than the whole construct's 0.872 because each scope's set is cut from fewer prompts: 0.59 of the lower ceiling, ten times chance. Across the twelve constructs whose token differs by scope the share runs from 0.44 (`Match`) to 0.71 (`Try`), and at the first checkpoint every share is lower: the two tokens' carrier sets converge over training without becoming one. Figure 2 shows where the carriers sit: ordered by how many of the 169 carrier sets a neuron belongs to, they concentrate on the neurons that carry many constructs, up to five times the 5% rate, and thin out towards neurons that carry few without stopping; and they sit in every one of the 24 layers. The full map of all 147,456 neurons, with the two scopes' sets drawn on it, is Figure C.1.

4.6 Shared neurons follow the inventory, not the grammar

When two constructs share carrier neurons, what do they have in common — a category of the inventory, or a place in the grammar? Their category in the inventory, not their place in the grammar. The carrier-set overlap of P_1's 14,196 pairs at the last checkpoint, clustered at 2 to 32 clusters and each inventory category scored for purity against 1,000 label shuffles, recovers grammar element against builtin name, the builtins' exception, warning, function and type classes, and one grammatical family, the comprehensions (purities and p-values in Table C.2). It does not recover statement against expression, compound statements, binding statements, or composites and their obliged parts. The highest-sharing pairs across categories pair a statement with the builtin that does the same job (`Delete`–`delattr`, `Global`–`globals`). The result does not come from the yardstick: with every carrier set rebuilt under a single pooled cell, every category that clusters significantly is still an inventory category and none a grammatical one (Appendix C.4).

4.7 A probe's accuracy saturates; the profile does not

Can a linear probe, the field's default instrument, track a construct across checkpoints as the profile does? On a trained checkpoint it cannot, because it separates everything. A linear probe on one layer at the read

site — residual stream for 164 constructs, hidden units for 31, folds holding out whole cores, shuffled-label controls at 0.50 — separates A from B on the untrained reference only partly: median accuracy 0.653 on the residual stream, 0.802 on the hidden units, and no construct reaches 0.99. On every trained checkpoint it separates them almost completely: median 0.998 at the first checkpoint, 0.999 at 84 B and at the end; 145–148 of 164 constructs at 0.99 or above; minimum 0.902. Accuracies span 0.902 to 1.000 with most at the ceiling, which cannot order 169 constructs, 24 layers or 61 checkpoints; the area under the curve is no better, and a harder contrast — two roles of one token — saturates too (Appendix C.5). The probe's margin, its one continuous reading, does move — a median 13.8 at 4.19 B, 16.5 at 83.9 B and 14.8 at the end (Appendix C.5) — but it is one number per construct and names no neurons; on the same two-role pairs the profile discriminates (§4.5), and its numbers move over training.

4.8 Three artefacts of the instrument, and the rules they produced

Which readings of this instance were wrong, and what rule replaced each? Three were, and each has been measured and is governed by a stated rule. Each is a trap for any reading of neurons across checkpoints, not only this one.

1. *The twin unit is too small for intervals.* The two twins share their cores, so their disagreement measures the variation of surface details (names, literals) and not the variation from choosing other cores; agreement corrected by the twin unit comes out above 1 for about 150 of 188 constructs. Rule: intervals come from the core bootstrap.
2. *The standardised size grows while the real size shrinks.* The yardstick's spread σ_0 shrinks over training, so the standardised norm rises for 144 of 169 constructs while the raw norm falls for 148. Rule: size is read in raw units.
3. *The layer of largest response is unstable.* Excluding the last layer moves it for 41% of constructs. Rule: comparisons are made in the product space, over all 147,456 neurons together, not layer by layer.

5 Where the profile stands

5.1 Against the alternative instruments

The situation. A researcher wants to know which neurons of a model carry a named construct, how reliably, and how that changes across hundreds of training checkpoints. The comparison below is limited to that situation. Outside it, several of these methods do things the calibrated concept profile cannot.

Probes. A probe is a classifier trained to read a property from the model's activations. It gives a direction in activation space and an accuracy. It does not say which neurons carry the property, it gives no chance level for comparing two constructs' carriers, and it is not causal; following a construct across checkpoints needs one probe per checkpoint. The whitened profile — the profile multiplied by the inverse covariance of the activations — is the mass-mean probe direction (Marks & Tegmark, 2023), so a probe of this kind is one view of the profile.

Sparse autoencoders and crosscoders. A sparse autoencoder is a second model trained to rewrite the activations as a sparse sum of learned features. It is trained on one checkpoint's activations. Features of two checkpoints are matched only by a crosscoder trained across them (Bayazit et al., 2026). The stimuli are the training corpus, so they are not declared, and a feature's activation has no noise floor; a feature is tested causally only by steering, as a separate step. Its strength is the one the profile lacks: it finds concepts nobody named. The two are complements. The profile tests a named concept with a unit; the dictionary proposes concepts to test.

Attribution and patching. These methods change one component's activation on one prompt and measure the effect on the output. They are causal, and they trace paths between components. They work per prompt, so they give no population statistic, and no unit for comparing checkpoints. The ablation of §4.4 applies their standard to the profile: the named neurons are removed and the loss at the construct's positions is read.

Neuron lists. Knowledge neurons are the neurons with the largest integrated-gradient attribution for a fact (Dai et al., 2022); language neurons are the neurons whose activation probability is most specific to one language (Tang et al., 2024). They are the profile's nearest relatives. They differ in three ways: the threshold has no unit, the reading is made at one checkpoint, and a neuron's ordinary behaviour at the position read is not removed; they are also rarely tested causally. The twin unit (§2.4), the yardstick (§2.3) and the licence (§2.5) are the additions. The turnover result of Wilam (2026) shows why they matter: a list read at one checkpoint describes that checkpoint.

Circuit-stability methods. Anani et al. (2026) certify each component's inclusion in a circuit as stable under bounded edits of the concept dataset, by randomised subsampling. Wu et al. (2026) measure how much a circuit changes when the prompts are resampled and when they are rephrased with other templates, and find that sparsity does not remove the template dependence. Their resampling and rephrasing variances are the counterparts of our core bootstrap and twins. Both certify circuits, made of components and edges, in trained models at one checkpoint. The profile certifies neurons, at every checkpoint.

What the profile adds, and what it does not claim. The calibrated concept profile gives, for a named construct, a reading of the model's neurons that is a measurement in the sense of §1.4: it states the four things a measurement must state.

1. *The population*, declared (§2.1).
2. *The unit*: the reading's spread when the battery is drawn again, the noise unit on every number (§2.4); each neuron's response is first stated in that neuron's own spread at the position read (§2.3).
3. *Evidence that its numbers come from the model*, from a licence run before any reading (§2.5, §4.1).
4. *For comparisons across checkpoints, the smallest change it can resolve*: the same units at every checkpoint and a measured resolution of 1,000 training steps, so one reading follows a construct through two trillion tokens (§4.2).

None of the other methods gives all four for a reading of neurons, and the profile trains nothing in between.

We read neurons, not features, because neurons are what training moves and the only units that can be followed across checkpoints without a second trained model. The carrier set is a measurement, not a claim that neurons are concepts.

6 Limitations

Of the method. The six choices the calibrated concept profile rests on, and how far each dependence is measured, are in §2.7 (Table 3); two are not measured at all — the reference corpus and the read point. Beyond them: it reads named constructs only, so it cannot discover concepts; and everything it says is relative to its distribution, so a neuron in no construct's carrier set may carry something the battery does not contain.

What changes in another setting. The method is not specific to code. The A/B/C design, the twins, the yardstick's principle (each neuron in its own spread at the same kind of position), the licence and the resolution do not depend on Python or on this model. What a new setting must supply: an inventory of named constructs fixed outside the model; a parser that decides a construct's role, or for natural language a tagger; a definition of the token out of its role, for set B (per word, in natural language); a reference corpus for the declared rates and for set C; a choice of cells for the yardstick, licensed again; and a read site, which for a model without MLP neurons (a state-space model, say) must be redefined. Everything in §3 is one realisation of these.

Of this instance. What the choices of Table 3 cost on OpenCoder — the gate, length sensitivity, the mixture, the companions — is stated in §3.1 and in §4.1's licence sentence. Beyond them: one model, one language — the battery is Python on a model that read 607 programming languages, so a construct's exposure in training is larger than its Python rate, and nothing here is shown on a second model. A battery that matches companions, and the natural arm of check 4, are what a new recording would add.

References

E. D. Abraham et al. CEBaB: Estimating the Causal Effects of Real-World Concepts on NLP Model Behavior. In *NeurIPS 2022*, 2022. arXiv:2205.14140.

A. Anani, T. Lorenz, B. Schiele, M. Fritz, and J. Fischer. Certified Circuits: Stability Guarantees for Mechanistic Circuits. In *ICML 2026*, 2026. arXiv:2602.22968.

D. Bau, B. Zhou, A. Khosla, A. Oliva, and A. Torralba. Network Dissection: Quantifying Interpretability of Deep Visual Representations. In *CVPR 2017*, 2017. arXiv:1704.05796.

D. Bayazit, A. Mueller, and A. Bosselut. Crosscoding Through Time: Tracking Emergence and Consolidation of Linguistic Representations Throughout LLM Pretraining. In *ACL 2026*, 2026. arXiv:2509.05291.

T. Bricken et al. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. *Transformer Circuits Thread*, October 2023.

Y. Chen, P. Qian, S. Wang, C. Peng, H. Xu, X. Wu, and Y. Sun. Noise Floor Audit for Agent Benchmarks, 2026. arXiv:2608.22331.

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse Autoencoders Find Highly Interpretable Features in Language Models, 2023. arXiv:2309.08600.

D. Dai et al. Knowledge Neurons in Pretrained Transformers. In *ACL 2022*, pp. 8493–8502, 2022. arXiv:2104.08696.

N. Elhage et al. Toy Models of Superposition. Anthropic, 2022. arXiv:2209.10652.

J. Gauthier et al. SyntaxGym: An Online Platform for Targeted Evaluation of Language Models. In *ACL 2020 System Demonstrations*, pp. 70–76, 2020.

M. Geva et al. Transformer Feed-Forward Layers Are Key-Value Memories. In *EMNLP 2021*, 2021. arXiv:2012.14913.

W. Gurnee et al. Universal Neurons in GPT2 Language Models. *TMLR*, 2024. arXiv:2401.12181.

J. Hewitt and P. Liang. Designing and Interpreting Probes with Control Tasks. In *EMNLP-IJCNLP 2019*, pp. 2733–2743, 2019.

S. Huang et al. OpenCoder: The Open Cookbook for Top-Tier Code Large Language Models, 2024. arXiv:2411.04905.

A. H. Kargaran, Y. Liu, F. Yvon, and H. Schütze. How Programming Concepts and Neurons Are Shared in Code Language Models. In *Findings of ACL 2025*, 2025. arXiv:2506.01074.

A. Karpathy, J. Johnson, and L. Fei-Fei. Visualizing and Understanding Recurrent Networks. In *ICLR 2016 workshop*, 2016. arXiv:1506.02078.

B. Kim et al. Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). In *ICML 2018, PMLR 80*, pp. 2668–2677, 2018.

J. Lindsey, A. Templeton, J. Marcus, T. Conerly, J. Batson, and C. Olah. Sparse Crosscoders for Cross-Layer Features and Model Diffing. *Transformer Circuits Thread*, October 2024.

S. Marks and M. Tegmark. The Geometry of Truth, 2023. arXiv:2310.06824.

M. Méloux, F. Portet, and M. Peyrard. Mechanistic Interpretability as Statistical Estimation: A Variance Analysis, 2025. arXiv:2510.00845.

T. Oikarinen et al. Evaluating Neuron Explanations: A Unified Framework with Sanity Checks. In *ICML 2025, PMLR 267*, 2025. arXiv:2506.05774.

A. Sharma, Z. Hu, C. J. Quinn, and A. Jannesari. Redundancy and Concept Analysis for Code-trained Language Models, 2023. arXiv:2305.00875.

D. Tan et al. Analysing the Generalisation and Reliability of Steering Vectors, 2024. arXiv:2407.12404.

T. Tang et al. Language-Specific Neurons: The Key to Multilingual Capabilities in Large Language Models. In *ACL 2024*, 2024. arXiv:2402.16438.

E. Voita, J. Ferrando, and C. Nalmpantis. Neurons in Large Language Models: Dead, N-gram, Positional. In *Findings of ACL 2024*, 2024. arXiv:2309.04827.

A. Warstadt et al. BLiMP: The Benchmark of Linguistic Minimal Pairs for English. *TACL*, 8:377–392, 2020.

P. Wilam. Representational drift in a code model: behaviour settles, which neurons carry it does not. Preprint, 2026.

F. Z. Wu, F. Tonin, and V. Cevher. Demystifying Variance in Circuit Discovery of LLMs. In *ICML 2026*, 2026. arXiv:2606.16920.

Z. Yin, X. Gu, and B. Shen. Neuron-Guided Interpretation of Code LLMs: Where, Why, and How?, 2025. arXiv:2512.19980.

A The realisation in full

A.1 The inventory and the reference corpus

1. *The inventory.* Every entry of Python 3.12's grammar that appears as text, and every builtin name: 63 + 143 = 206 constructs. 188 were built; 18 were not (item 3).
2. *What the rule excludes.* Of Python 3.12's grammar nodes, 48 never appear as text of their own, and are left out: module containers, context markers, pure wrappers, product types, and the operator markers that fold into `BinOp`, `BoolOp`, `Compare` and `UnaryOp`. That leaves 63. The builtins are every public name in `builtins` except the six the interactive shell injects (`help`, `exit`, `quit`, `copyright`, `credits`, `license`).
3. *The eighteen not built.* Seven have a corpus rate but admit no family of cores, because several grammar nodes share one surface token (`None` three times, `yield` twice, and `await`, `{`, `f"`). The other eleven are the syntax Python added in versions 3.10–3.12 — the six `match` patterns, `except*`, and the three type parameters and the type alias of PEP 695 — which the reference corpus does not contain often enough to calibrate.
4. *The reference corpus.* 253,398 parsed Python files. The written list of factors and rates drawn from it is ν, and ν's rule is that every property of a prompt is set to the corpus rate, never chosen for a reading.

A.2 Sets, cores, twins, scope

1. *Sets A and B.* 720 prompts per set in each twin, ninety from each of eight cores, for every one of the 188 constructs; a set across both twins is 1,440 prompts.
2. *Cores.* Drawn from a family whose size is the product of the construct's axes, from 42 (`Starred`) to 1,280 (`For`), median 210 (`CORE_COUNTS.md`).
3. *B's sub-sets.* One per kind of out-of-role use — docstring, comment, string, identifier — and, for the 18 constructs whose token is shared with another construct (`for`: `For`, `AsyncFor` and the four comprehensions; `def`, `with`, `if`, `:`, `{`, `[`), one with the token in the other construct's role, built from that construct's own cores: 4,850 prompts, 1.8% of B (`B_SUBTYPES.md`). No rendered prompt appears in both a construct's A set and another construct's B set.
4. *Set C.* One pool for all constructs, generated from the same menus and the same formatter as A and B with no construct anchored in it, and longer: 97.7 tokens on average against A's 56.9.
5. *Scope.* Every recorded quantity carries the prompt's scope — module, function or method — because one construct is read through two different tokens depending on where it stands (§3.1, §4.5).
6. *In all.* 569,270 prompts and 33,271,350 tokens, 58.45 tokens per prompt (median 55, quartiles 28–81).

A.3 How one prompt is made

Four stages, each fixing one thing. (1) *The core fixes the construct's own shape.* A core is one combination of the construct's axes; for `for` the axes are the operand, the suite, the access and the body, and the family is every combination, 1,280. Eight are drawn. Core c3 of `for` is {operand: attribute, suite: two statements, access: bare, body: early exit}; core c2 is {operand: `range`, suite: companion, access: subscript, body: count}. (2) *The cell fixes the surroundings.* A cell is (domain, scope, padding, draw): five vocabulary domains (finance, biology, gaming, physics, e-commerce) × three scopes (module, function, method) × three padding levels × two draws = 90; every cell appears under every core, so 8 × 90 = 720 prompts a set. (3) *The menu fills the names.* Identifiers and literals are drawn at corpus rates and injected into the slots the core declares; the core never chooses a word. The same core and cell drawn twice give the same shape with every name new (Figure A.1):

```
# For/c3, module scope, e-commerce, draw 1
requirement_val = next_indent_head.run_command
error_level_ok  = repo_name_total > 0
for threading_list in ParseNolintSuppressions_store.ParseNolintSuppressions:

# For/c3, draw 2
coupon_redemption_val = STUDENT_VIEW_head.NumLines
if carriage_return_found_ok:
    PrintUsage_all(commentpos)
for newpath in closing_brace_pos_store.closing_brace_pos:

# For/c2, module scope, biology
click_val = overlap_head.statements
for iterator in range(3):
    if ligand_ok:
        transcript_total += iterator * 1
```

Figure A.1: Three prompts from the `for` battery: core c3 drawn twice (module scope, e-commerce), and core c2 (module scope, biology). The shape is the core's; every name is the menu's.

(4) *Set B.* B's prompts are built from their own shapes, which place the token in a docstring, a comment, a string or an identifier — and, for the 18 token-sharing constructs, in the sibling construct's role (for `for`: `[(d, 2) for k, d in zip(...)]`).

(5) *The companion cap.* B's companion rates are fitted so that B's companion mix matches A's; the cap is the check that it does.

(6) *Format, parse, read site, log.* Every prompt is formatted with `black` and parsed, and its construct's node is located. The read site is the node's first token. Every draw is logged with the prompt. The generator's own rule, in its source: "data in the loop is fine; the measured model in the loop is never".

(7) *One asymmetry between A and B.* A's grid is balanced by construction: 240 prompts per scope and 240 per padding level. B's is not: for `for`, 219/234/267 by scope and 175/262/283 by padding, because B's shapes do not all admit every cell. The yardstick standardises each read site in its own cell, so the imbalance does not enter the profile; a cell-by-cell comparison of A and B should expect it.

A.4 The read-site gate

A construct is read only if its A and B prompts land on the same token piece in at least 80% of prompts, in both twins, on the indented scopes. Named builtins: 143 pass, none fail. Structural constructs: 1 passes, 14 fail — the operator families, the symbol-anchored expressions and `Assign`, `AnnAssign` — because the tokenizer merges a symbol with its neighbours. Keyword tokens: 25 pass, 5 fail because the piece at the read site is not one piece — `FunctionDef`, `ClassDef`, `AsyncFunctionDef` split `␣def/def` (`␣class/class`)

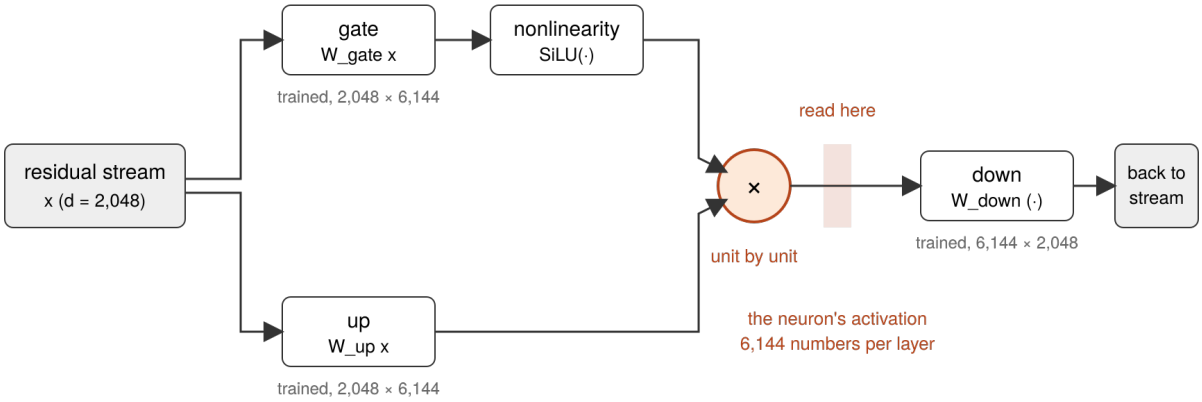


Figure A.2: The read point in one MLP block (SwiGLU): the neuron's activation is the gate branch after the nonlinearity times the up branch, taken before the down-projection.

across B's subtypes, `BoolOp` reads `␣and` or `␣or`, `UnaryOp` reads `␣not`, `␣~` or `␣-`. The gate is the realisation's dependence on its tokenizer, and the first thing a port to another model must re-check.

A.5 The neuron that is read (Figure A.2)

One MLP block of OpenCoder-1.5B. The residual stream (width 2,240) enters two trained projections, the gate (2,240 × 6,144) followed by the SiLU nonlinearity, and the up (2,240 × 6,144); their outputs are multiplied unit by unit, giving 6,144 numbers, the neurons' activations; the trained down-projection (6,144 × 2,240) returns the product to the residual stream. The profile reads the 6,144 numbers after the product and before the down-projection, the quantity Geva et al. (2021) read as a key-value memory. The three matrices are the ones whose steps over training Wilam (2026) measures.

A.6 The record

What is stored. Per reference, for every construct, both twins, every set and all 24 layers: per-core accumulators of the count and the mean raw activation per neuron, with the cell axis kept; the yardstick; and the loss at every offset of every prompt, which the ablation of §4.4 reads. The accumulators take 27.6 GB a reference; the four per-prompt references are 107 GB each; 2,022 GB in all. What is stored per prompt is in §3.2.

Per prompt. The 31 constructs with per-prompt MLP activations were drawn so that the checks that need them license the population that is read: 15 constructs the design requires (the pilot's and the two-use pairs' members that pass the read-site gate) and 17 builtins, in the read population's proportion of one-piece and multi-piece tokens. One planned construct (`tuple`) is missing at all four references. Storing the neurons per prompt for every construct would have cost about six times as much: 890 GB across the four references instead of 147 (decimal gigabytes, the per-prompt neuron arrays alone; `RECORD_SIZE.json`).

A.7 The counts

Table A.1: The counts of the battery and the record.

object	count
inventory: grammar entries + builtin names	63 + 143 = 206 constructs
built / not built	188 / 18
pass the read-site gate (P_1)	169 of 188
gate failures: structural / keyword tokens	14 / 5
reference corpus	253,398 parsed Python files
prompts per A and per B set, per twin	720 (8 cores × 90), all 188 constructs
cores per construct	8, from a family of 42–1,280 (median 210)
prompts, tokens	569,270; 33,271,350
tokens per prompt	mean 58.45; median 55; quartiles 28–81
yardstick cells	20 (4 token classes × 5 fifths of the prompt)
references recorded	61: 35 pretraining, 24 annealing, 2 untrained
storage	27.6 GB of accumulators a reference; 107 GB at each of the four per-prompt references; 2,022 GB in all

A.8 Choices this realisation made, and their alternatives

Each is a choice §6 lists as a property of the instance, not of the method.

1. *Inventory:* the grammar entries and builtin names of Python 3.12. Alternatives: another language and its grammar; a curated subset.
2. *Reference corpus:* public Python, since the model's training data was not released. Alternative: the training corpus, where released.
3. *Cores and twins:* eight cores, shared by the two twins. Alternatives: more cores with fewer prompts each, which trades surface noise for core noise; twins with independent cores, which would make the twin unit carry both (§4.8, artefact 1).
4. *Read site:* the first token of the construct's node, on its MLP neurons. Alternatives: a later token of the construct; the residual stream; a read site that does not depend on the token piece. Each changes what is read, not the arithmetic.
5. *Cells:* token class × fifth of the prompt, 20 cells. Other splits are possible, under the two requirements of §3.3.

The generator's build — the grammars, the calibration tables, the companion mechanism, the matched control sets, the loss positions and the amendments between design and battery — is documented with the code (Appendix E).

B The licence, construct by construct

The checks of Table 4 at the final pretraining checkpoint (`pretrain_iter_0484865`, 2,034 B tokens); checks 2, 3, 7a and 7b after the re-run of 19 October (Appendix D). B.1 gives each check's spread over the constructs, B.2 the constructs the checks single out, B.3 the companion check and B.4 length sensitivity. The values for every construct are released as `TABLE_B1.csv`, `TABLE_B2.csv` and `TABLE_B3.csv`.

B.1 Each check over the constructs

Table B.1: Each check over the constructs it ran on: the median, and the lowest and highest values with the constructs that hold them. The gate and checks 1, 4, 5 and 8 ran on all 188; their medians over the 169 that pass the gate are in brackets. Checks 2, 3, 7a and 7b ran on the 31 with per-prompt activations. Check 6 is in B.3.

check	what is read	ran on	median	lowest	highest
gate	share of prompts whose A and B read sites land on the same token piece (pass: 0.80 or above)	188	169 pass, 19 fail (B.2)	0.250 (`BinOp`, `Call`, `Compare`, `List`); among those that pass, 0.873 (`NamedExpr`)	1.000
1 twin agreement	cosine between the two twins' profiles	188	0.9861 (0.9889)	0.7419 (`UnaryOp`); among the 169, 0.8771 (`None`)	0.9980 (`NamedExpr`)
2 convergence	prompts needed for a cosine of 0.99 with the profile on all 720, read on a grid of 8, 24, 56, 120 and 176	31	56	24 (`Delete`, `NotImplemented`, `With`, `ord`)	120 (`For`, `If`, `ImportFrom`, `ResourceWarning`)
3 length index	the length index of the profile	31	0.75	0.41 (`IfExp`, `ListComp`)	3.36 (`With`)
3 flagged share	share of the construct's neurons flagged as length-sensitive	31	0.024	0.002 (`IfExp`, `SetComp`)	0.554 (`With`)
4 re-weighting, cosine	cosine between the profile with the logged factors re-weighted and the base profile	188	0.9095 (0.9135)	0.6923 (`BinOp`); among the 169, 0.7852 (`Pass`)	0.9943 (`Global`)
4 re-weighting, twin units	the distance between the two over the construct's own twin distance (Appendix D)	188	2.30 (2.35)	0.71 (`Global`)	4.94 (`bytearray`)
5 calibration	factor by which the core-bootstrap intervals understate the twins' difference (1 is calibrated; above 1, too narrow)	188	0.997 (1.005)	0.45 (`callable`, `format`, `zip`)	7.17 (`NotImplemented`)
7a scrambled null	the real profile's norm at layer 12 over the scrambled-label null's 95th percentile (pass: above 1)	31	10.66	7.22 (`With`)	13.01 (`ord`)
7b split-half null	the same over the split-half null's 95th percentile (pass: above 1)	31	12.76	5.96 (`With`)	15.83 (`NamedExpr`)
8 yardstick floor	shift of the profile when the floor on σ_0 is set three ways, in twin units	188	0.0039 (0.0041)	0.0008 (`Attribute`, `UnaryOp`); among the 169, 0.0013 (`None`)	0.0347 (`Nonlocal`)

All 31 pass checks 7a and 7b.

B.2 The constructs the checks single out

The gate. 19 constructs fail, with the share of prompts on one piece in brackets: `AnnAssign` (0.617), `Assign` (0.406), `AsyncFunctionDef` (0.518), `Attribute` (0.375), `AugAssign` (0.375), `BinOp` (0.250), `BoolOp` (0.625), `Call` (0.250), `ClassDef` (0.529), `Compare` (0.250), `Dict` (0.371), `FunctionDef` (0.503), `List` (0.250), `Set` (0.375), `Slice` (0.500), `Starred` (0.500), `Subscript` (0.542), `Tuple` (0.371), `UnaryOp` (0.375). Of the 169 that pass, all but two are at 0.995 or above; the two are `NamedExpr` (0.873) and `Ellipsis` (0.992). Why they fail is in Appendix A.4.

Check 3. Eight statements have more than 5% of their neurons flagged as length-sensitive: `With` 55%, `For` 49%, `If` 47%, `While` 36%, `ImportFrom` 31%, `Assert` 21%, `Delete` 10%, `Return` 8%. B.4 reads them with the length held.

Check 4. Four constructs sit within one twin unit of their base profile: `Global` (0.71), `UnaryOp` (0.74), `Continue` (0.77), `Attribute` (0.99). Two of them, `UnaryOp` and `Attribute`, fail the gate.

Check 5. 33 constructs have intervals understated by more than 1.5, 30 of them among the 169 read: `NotImplemented` 7.17, `Exception` 5.44, `KeyboardInterrupt` 4.25, `EnvironmentError` 3.83, `ConnectionAbortedError` 3.76, `PendingDeprecationWarning` 3.68, `ModuleNotFoundError` 3.63, `ResourceWarning` 3.53, `IfExp` 3.24, `GeneratorExp` 2.86, `Pass` 2.86, `ListComp` 2.86, `SetComp` 2.72, `DictComp` 2.47, `Try` 2.20, `globals` 2.19, `AsyncFor` 2.12, `isinstance` 2.10, `iter` 2.09, `AsyncWith` 2.05, `Assert` 2.03,

Table B.3: Check 6, for the eight constructs: the concept norm at layer 12, and the asymmetry — as an L2 norm over all companions and as the largest single one (max) — counted over every line of every prompt (whole) and over the prompts drawn without padding (pad-0). The construct's own components, as the inventory lists them, are excluded.

construct	concept norm (layer 12)	L2 whole	L2 pad-0	max whole	max pad-0	companions
ClassDef	53.06	1.064	1.311	0.388	0.646	33
For	50.54	0.644	0.780	0.228	0.358	36
FunctionDef	36.12	0.812	0.863	0.530	0.526	34
If	44.93	0.919	0.989	0.678	0.688	35
Match	47.65	0.395	0.464	0.186	0.261	32
Try	70.75	0.572	0.729	0.266	0.524	33
While	72.57	1.043	1.187	0.498	0.542	34
With	55.46	0.693	1.030	0.353	0.667	36

`Delete` 1.95, `Match` 1.95, `ord` 1.92, `hex` 1.85, `divmod` 1.85, `ascii` 1.77, `Return` 1.73, `memoryview` 1.68, `While` 1.67. The three that fail the gate: `ClassDef` 2.00, `AsyncFunctionDef` 1.97, `FunctionDef` 1.75.

B.3 The companion check

Check 6, on the eight constructs that can have a companion. The asymmetry is the difference between A's and B's share of the prompts that carry a companion; it is a count over prompt text, measured on the battery the model was run on, and has no layer.

Why two readings. The record does not keep the boundary between a prompt's padding and its core. *Whole* counts every line; *pad-0* counts only the prompts drawn without padding, where the boundary is the first line and the count is exactly the generator's — a third of each A set by design, 14–35% of each B set. Companions that are the construct's own components, as the inventory lists them, are excluded; the re-weighting of §4.1 excludes the same ones. The generator's audit uses a shorter list, which differs for four of the eight (`For`, `While`, `Try`, `Match`) and changes only `Match` materially: its largest difference is 0.186 here and 0.980 under the audit's list (both in `CHECK6_COMPANIONS.json`).

Where the difference comes from, and where it can be removed. Some of the difference is what the construct's grammar requires (every `match` case ends in `pass`; an `if` test is usually a comparison); some comes from how B was built (B's strings are f-strings; its sibling-role prompts are drawn from other constructs, the async variants among them). B's prompts are ordinary code with the token out of role and carry companions at a fifth of prompts or so, not near zero. Re-weighting B to A's companion shares needs per-prompt activations: for `For` and `With` the shares are attainable and the shift is measured (§4.1); for `If` and `While` no weighting of the drawn prompts attains them (`While`'s B contains a comparison in every prompt, against a share of 0.69 in A); for `ClassDef`, `FunctionDef`, `Match` and `Try` no per-prompt activations exist.

Which neurons are named, and why the re-weighting is not adopted. `With`'s recorded and re-weighted carrier sets overlap by 0.77 at the final checkpoint, below the band of the two conditions' twin ceilings (0.92–0.94), and likewise at the other trained references; for `For` the two sets agree within that band at every reference. The re-weighting rests on a fifth to a third of the prompts and lowers the twin ceiling at every reference (`For` by 0.05–0.12, `With` by 0.02–0.13), so the companion difference is stated as a restriction rather than corrected.

B.4 Length sensitivity under the yardstick

For the eight length-sensitive statements of B.2, padding is added before the construct, so it moves the read site to a later fifth of the prompt in 54–98% of prompts across the eight, and each padding level is then standardised in a different cell. Held in its unpadded cell, the length index falls by a median 26% and the flagged share by 41%; `Delete` and `Return` fall below 5%, and six remain — `With` 40%, `For` 31%, `If` 31%,

Table C.1: Loss rise over the random mean, nats (constructs above, of 16).

reference	20-cell split	one pooled cell
4.19 B tokens	+0.08 (13)	+0.25 (15)
83.9 B	+0.27 (14)	+0.51 (14)
2,034 B	+0.40 (14)	+0.73 (14)

`While` 19%, `ImportFrom` 16%, `Assert` 14%. The direction of the profile is largely kept: the lowest cosine between padding levels rises to 0.88–0.99 (`With` 0.79 → 0.97).

C Supporting readings for §4

Each item names the section it supports: C.1 the ablation and the yardstick (§4.3, §4.4); C.2 the running example over training (§4.2, §4.8); C.3 the map behind Figure 2 (§4.5); C.4 the shared neurons (§4.6); C.5 the probe (§4.7).

C.1 The ablation comparison across yardsticks

Each yardstick names a carrier set per construct; the set is ablated and the loss rise is read over the mean of 20 size-matched random sets, in nats. Medians over the 16 position-marked constructs, with the number of constructs above the random mean (sign test). `Try` is included: its carriers cost 5.95 nats more than a random set on a 0.088-nat baseline.

The 20-cell split gives the lower effect at every checkpoint, as expected: it excludes neurons that are large only because of where they stand, and ablation rewards size. The 20-cell figures are from a 20-mask run; §4.4's are from the 200-mask run of the same experiment (`TABLE_C3_yardsticks.md`).

Why the comparison is with the random mean, in nats. The random sets' spread is long-tailed at the two later references — a few random sets hit load-bearing neurons — and it keeps growing with more draws, so it does not settle into a usable unit (§4.4).

Per construct. Taken one at a time against 320 random sets, 2 of 16 constructs exceed every random set at 4.19 B tokens, 8 at 84 B and 8 at the end, and 2, 8 and 8 remain clear at 0.05 after a Holm correction. The floor of 1/321 is the resolution a 320-set control allows; the paper's claim is the one over constructs.

C.2 `for` at four references

Untrained, 4.19 B, 83.9 B and 2,034 B tokens. The running example's numbers over training, behind §4.2; the raw norm falls from 4.19 B on while the standardised norm rises to 83.9 B and ends above its 4.19 B value, which is artefact 2 of §4.8 seen on one construct. Raw norm 15.7 → 184.5 → 143.0 → 96.0. Standardised norm 30.2 → 239.6 → 270.9 → 251.5. Sparsity 0.390 → 0.471 → 0.538 → 0.505. Twin agreement 0.427 → 0.991 → 0.984 → 0.984. Depth 10.94 → 11.89 → 11.09 → 10.90. Twin ceiling 0.293 → 0.902 → 0.889 → 0.872. The two twins agree almost perfectly from the first checkpoint on.

C.3 The full neuron map (Figure C.1)

In Figure C.1, all 24 × 6,144 MLP neurons of OpenCoder-1.5B drawn in place at the end of pretraining, one row per layer, the neurons within a layer ordered by degree as in Figure 2, the order fixed once and reused.

C.4 Shared neurons by category (§4.6)

What was clustered, and why. Two constructs share neurons when their carrier sets overlap. The question is whether the constructs that share most fall into the grammar's categories or into the inventory's. The overlaps of P_1's 14,196 construct pairs at the last checkpoint were clustered into 2 to 32 clusters, and each

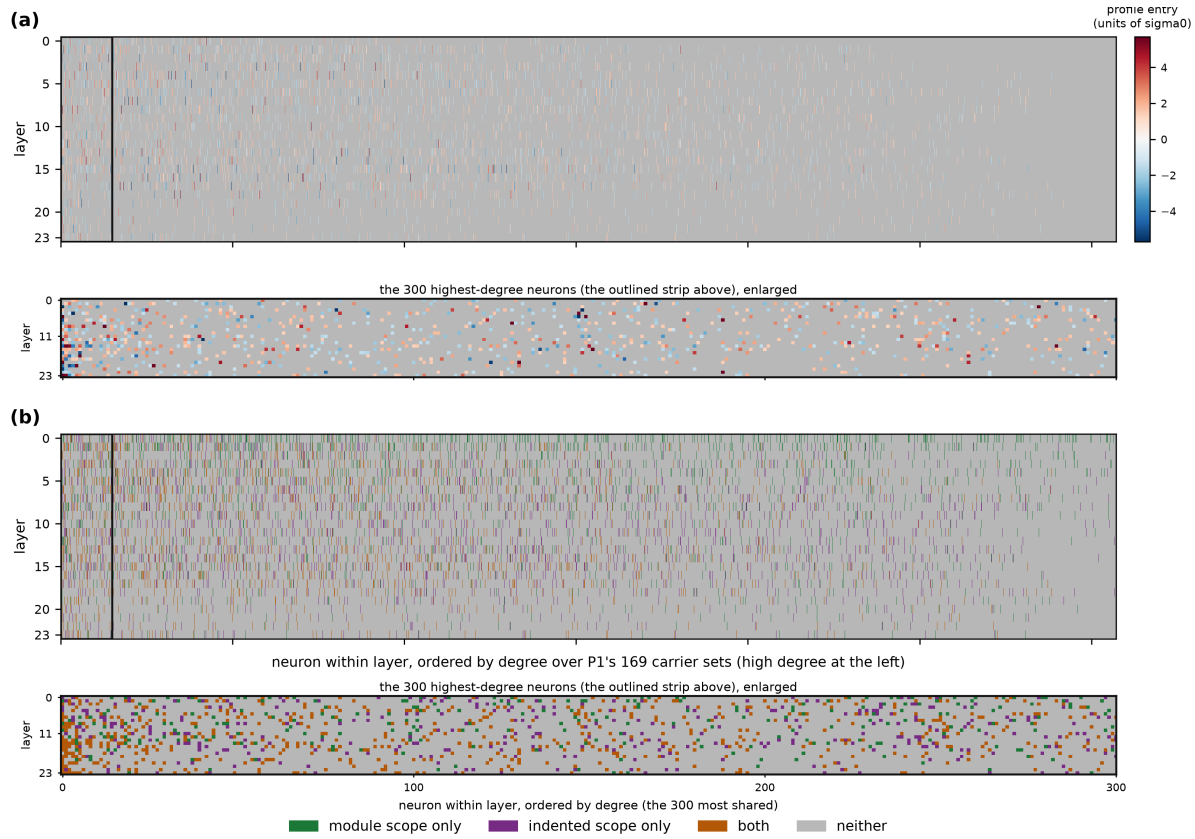


Figure C.1: Panel (a): the `for` loop's carrier set, each carrier in the colour of its entry $\pi(\mathrm{A}) - \pi(\mathrm{B})$ in units of σ_0, every other neuron grey; an inset enlarges the 300 highest-degree neurons so that single entries can be read. Panel (b): one role, two tokens — the indented ␣`for` and the module-level `for`, two unrelated vocabulary entries; each neuron coloured by which of their two carrier sets it belongs to (module only, indented only, both, neither), with the same inset. What the map shows: the carriers are densest at the high-degree end and present at nearly every degree, falling to none among the lowest-degree neurons, and present in every layer; in panel (b) the shared neurons (both) and the scope-specific ones are interleaved everywhere, not grouped by layer or by degree (their overlap is given in §4.5). What it does not show: individual entries outside the inset, or any layer pattern finer than presence in all 24.

category was scored for purity — the share of a cluster's members that belong to the category — against 1,000 shuffles of the labels.

Not recovered: statement against expression, compound statements, binding statements, composites and their obliged parts.

Other readings. The highest-sharing pairs across categories are `Delete`–`delattr` (0.200) and `Global`–`globals` (0.094). Keyword tokens sit at a median depth of 10.98 layers and builtins at 11.97, with indistinguishable concentration (sparsity 0.666 against 0.673). Under one pooled cell, 38 of 189 purity tests (7 cuts × 6 categories, Holm-corrected) are significant, all of them inventory categories (`POOLED_SETS_45.json`).

C.5 The probe in full (§4.7)

The area under the curve: a median 1.000 at every trained reference, 145–148 of 164 constructs at or above 0.99. Two roles of one token (`for` as loop against `for` in a comprehension) read 0.68–0.75 untrained and 0.999 at the median from the first checkpoint on. The probe's margin, the signed distance of a held-out prompt from the boundary in the probe's own units, moves: a median 3.2 untrained, 13.8 at 4.19 B, 16.5 at

Table C.2: Categories recovered by the clustering ($p < 0.001$ unless noted).

category	constructs	purity
grammar element against builtin name	—	recovered
exception classes	69	1.000
warning classes	12	1.000
builtin functions	43	0.86
builtin types	26	0.73
comprehensions	4	1.000 ($p = 0.004$)

83.9 B and 14.8 at the end, against a control margin near zero throughout. It rises to 83.9 B for 128 of 164 constructs and falls again to the end for 149 of 164 (paired sign tests, $p < 10^{-12}$), ending 7% above its value at 4.19 B — a turn at the same point where the raw size of the response begins to fall (Wilam, 2026, §4.4).

D Reproducibility

Every figure in §3 and §4 is read from the record or from the battery's own files as stored, not estimated; where a figure is an approximation the text says so. Battery hash `570b42db...bcbc1ad`. The release: the dataset `piotrwilam/AtlasOpenCoder`, tag `v1`, DOI 10.57967/hf/11472 — the battery in full, the carrier sets at every reference, the licence tables, the readings behind the figures, and a worked example of the record and of the profiles for `for` — and the repository `piotrwilam/AtlasCalibratedConceptProfiles` (the generator, the recorder, the checks and every script of Table D.1). The dataset's `MANIFEST_PAPER1.md` lists every number in this paper with the file and field it is read from and the script that rebuilds it.

Where each number comes from. Each table and figure, the stored file it is read from, and the script that wrote the version in this paper. Files are in `figures/` and `figures/data/`; scripts in `experiments/`.

Table D.1: Where each figure and table comes from.

in the paper	file	script
Tables 1–3 (the factors, the nine checks, the six choices)	— (text)	—
Table 4 (the licence as run)	`LICENCE_TABLE.json`, `APPENDIX_B1_checks.md`	`licence_table.py`, `licence_tables.py`
Appendix B.1–B.2, the gate and checks 1, 4, 5, 8	`TABLE_B1.csv`	`licence_tables.py`
Appendix B.1 and D, check 4 in twin units	`CHECK4_RELATIVE.{csv,json}`	`check4_relative.py`
Appendix B.1–B.2, checks 2, 3, 7a, 7b	`TABLE_B2.csv`; `RECHECK_PER_PROMPT.json` (check 2's prompt counts)	`licence_tables.py`
Appendix B.1, the two null ratios	`NULL_RATIOS.json` (the layer-12 real norm, each null's p95 and the ratio)	`licence_tables.py`
Appendix B.1–B.2, the gate	`GATE_SHARES.json` (the four shares per construct and their minimum)	`licence_tables.py`
§2.6, §3.3 and A.8, the twin ceilings	`TWIN_CEILINGS.json` (every construct at all 61 references, both spaces)	`twin_ceilings.py`
Appendix B.3 (companion)	`CHECK6_COMPANIONS.json` (`table_b3.rows`), `TABLE_B3.csv`	`check6_companions.py`, `licence_tables.py`
Appendix B.4 (length, band held)	`LENGTH_BAND_HELD.json`	`band_held.py`
Figure 1 (the method in six steps)	`FIG1_method.{png,svg,json}`	`figure_method.py` — a schematic; nothing in it is measured
Figure 2 (`for`'s carriers in place)	`FIG1_carriers_in_place.{png,svg,json}`	`figure1_small.py`
Figure A.1 (three prompts from the battery)	`phase_battery/twin1/For_A.jsonl.gz`	none: quoted from the battery files
Figure A.2 (the neuron read)	`FIG_neuron_swiglu.{png,svg}`	none: a schematic, drawn by hand; it shows no data

continued

in the paper	file	script
Figure C.1 (the full neuron map)	FIGC1_neuron_map.{png,svg,json}	figure_c1.py
§3.1 core families	CORE_COUNTS.{json,md}	core_counts.py
§3.1 B's sub-sets	B_SUBTYPES.{json,md}	b_subtypes.py
§4.2 resolution	B10_SPACING_ADJUSTED.json	b10_fit.py
§4.3 coarser yardsticks	TURNOVER_CLASSES_ONLY.json	b10.py
§4.4 ablation	ABLATION_NATS.json, TABLE_D1_ablation.{md,csv}	ablation_ranks.py for the version in the paper (it writes the rank and exact-p columns and removes control units), reading ablation_full.py's output; ablation_stable.py and ablation_nats.py also write TABLE_D1_ablation and give different numbers
§4.5 two roles of one token	TWO_USE_SHARING.json	atlas.py b5
§4.5 one role, two tokens	SCOPE_OVERLAP.json	atlas.py b4
§4.6 and C.4 shared neurons by category	FAMILIES.json	atlas.py b6
§4.6 the pooled-cell replication	POOLED_SETS_45.json	pooled_sets_169.py
§4.7 probe margin	PROBE_MARGINS.json, TABLE_probe_margins.{md,csv}	probe_margins.py, reading PROBE.json from b9_b10.py
§4.8 the three artefacts	LICENCE_TABLE.json (1, the twin unit against the core bootstrap), FOR_EXAMPLE.json (2, standardised against raw size), FAMILIES.json (3, the layer of largest response)	licence_table.py, atlas.py b3, atlas.py b6
Appendix C.1 (yardsticks, in nats)	C3_YARDSTICK_NATS.json, TABLE_C3_yardsticks.{md,csv}	c3_yardstick_nats.py
Appendix C.2 (for at four references)	FOR_EXAMPLE.json	atlas.py b3
Appendix C.5 (the probe in full)	PROBE.json, PROBE_TWO_ROLE.json	b9_b10.py
Appendix D hashes	PROVENANCE.md, IRREPLACEABLE.json	build_provenance.py, irreplaceable.py
Appendix D, the licence re-run	LICENCE_TABLE.json (as published), LICENCE_TABLE_CORRECTED.json (the 19 Oct re-run)	licence_table.py
§3.5 watched numbers	record_GATE_111.json	profilebattery/validation/watched.py
A.6 storage	RECORD_SIZE.json	record_size.py

Every numbered figure and table in the paper, and every section of §4 and Appendix C that reports a measured number, has a row above; each named file was checked to exist in the release tree before the row was written. Two rows name no script, and say why: Figure A.1 quotes three prompts straight from the battery files, and Figure A.2 is a hand-drawn schematic that shows no data.

The untrained reference. Its random weights do not regenerate across library versions, and the library versions used for its two recordings were not stored; the recorded profiles are kept, and a reproducible seed is owed.

Licence re-run. Four checks were re-run on 19 Oct 2026 after a fault in how per-prompt vectors were joined to their prompts was found and fixed (the files are stored in the recorder's length-sorted order, not in jsonl row order). Convergence was unchanged at the median (56 prompts for a cosine of 0.99). Length sensitivity was lower than first reported (median index 0.7455 against 1.2142; flagged share 2.4% against 3.9%; the two earlier values are from the pre-fix run's report and are not in a released artefact, since LICENCE_TABLE.json kept only the length levels and the minimum pair cosine). The re-run also drew 20 null replicates per construct, against 8 before. Both nulls fell. *Their centre* — the null profile's norm at layer 12, in the profile's own units, the median over replicates and then over the 31 constructs — went from 4.86 to 4.57 for the scrambled null and from 5.27 to 4.64 for the split-half. *Their tail* — the 95th percentile, against which §4.1's margin is taken — fell further for the split-half: from 6.42 to 6.13 (−4.5%) for the scrambled null and from 6.45 to 5.30 (−18%) for the split-half. The margin above the tail, the real profile's norm over the null's 95th percentile (§4.1), therefore rose: from 10.1× to 10.7× for the scrambled null and from 9.6× to 12.8× for the split-half. It had been understated, not overstated; quoting the centres alone would understate how much more the split-half margin moved.

Check 4 in twin units. The licence table kept check 4's cosine and not its distance, so Table B.1's twin-unit row is rebuilt from both arms' per-core, per-cell accumulators: the median over the 188 is 2.30 against the licence suite's 2.34, and 4 of 188 sit within one twin unit, as §4.1 states (`check4_relative.py`, `CHECK4_RELATIVE.csv`). A second route, from the stored cosines through the chord identity $d = \sqrt{(2(1 - \cos))}$, holds per pair but not between medians: on check 1, where both forms are stored, it is within 5% for 90% of constructs and at worst 18.6% off, and it gives a median of 2.25 (quartiles 1.82–3.49), with 7 of 188 within one twin unit.

E Applying the method

For a reader who wants to run the method on another model or language. The steps are those of §2; the costs are this realisation's.

1. *Inputs.* A model with at least one checkpoint (one checkpoint gives a reading; a series gives readings across training). An inventory of named constructs fixed before the model is read. A reference corpus in the language, for the declared rates and for set C. A parser or tagger that decides each construct's role, with a surface token per construct. Model access that returns activations at a named site. (§6, *What changes in another setting.*)
2. *Declare the distribution.* List the factors around a construct (identifiers, literals, scope and nesting, companions, length, surrounding code) and measure each one's rate on the reference corpus. Here: codeparrot-clean, 253,398 parsed Python files.
3. *Generate the battery.* For each construct, set A (the construct in its role) and set B (its token out of role), the same companions in both, drawn twice — the twins — from eight cores per construct; set C from the reference corpus. Log every draw; hash the battery. Here: 188 constructs, 720 prompts per set and twin, 569,270 prompts in all.
4. *Gate the read site.* Keep a construct only if its A and B prompts put the read token on the same piece in at least 80% of prompts (Appendix A.4). Here: 169 of 188 pass.
5. *Record.* Run the model on the battery at each checkpoint; store per-core means of every neuron's activation, with the cell axis kept, so the yardstick and the cut can be changed later; store per-prompt activations for a subset if checks 2, 3, 7a and 7b are wanted. Here: about 28 GB of per-core means per reference, and 79 GB more at a reference with per-prompt arrays (36 GB for the neurons of 31 constructs, 43 GB for the residual stream of 164).
6. *Compute the calibrated concept profile.* Per neuron: the mean on A minus the mean on B, each in the neuron's own spread on C in the same cell; the twin difference gives the noise unit.
7. *License.* Run the nine checks of Table 2; the two nulls must pass; report the other seven (Table 4 is the form).
8. *Read.* The profile, the carrier set and its twin ceiling at each checkpoint; across checkpoints, the resolution first (§2.6), then any date or comparison in twin units.
9. *Check by removal.* Ablate the carrier set and a random set of the same size, and read the loss at the construct's positions (§4.4).

Code. Five stages, each a script in `experiments/` of the repository `piotrwilam/AtlasCalibratedConceptProfiles`.

The stage table and its scripts are released with the repository; a minimal example, running stages 3–5 and the profile for one construct on one small model, will follow.

Table E.1: The five stages, their scripts and their outputs.

stage	script	output
1. inventory	`split_p1.py`	`inventory/inventory_v2.json`, `validation/battery/SPLIT.json`
2. corpus calibration (the declared list ν)	`build_calibration.py`	`calibration/*.json`
3. battery	`build_battery.py`	`phase_battery/`, `CONTENT_HASHES.json`
4. recording	`run_s15.py`, `run_s16.py` (`run_colab.py` for a hosted GPU)	`record/<ref>/`
5. licence	`licence_table.py`, `licence_tables.py`	`LICENCE_TABLE.json`, Tables B.1–B.3